

AAPS 31/05/2025

Bibliotecas Python Mais Utilizadas Atualmente

1. NumPy

Explicação: Biblioteca para computação numérica eficiente com suporte a arrays multidimensionais e operações matemáticas.

```
import numpy as np

# Criando um array NumPy
arr = np.array([1, 2, 3, 4, 5])

# Operações matemáticas
print(arr * 2)          # [ 2  4  6  8 10]
print(np.mean(arr))     # 3.0
print(arr.reshape(5, 1)) # Transforma em matriz 5x1
```

2. Pandas

Explicação: Biblioteca para manipulação e análise de dados, com estruturas como DataFrames.

```
import pandas as pd

# Criando um DataFrame
data = {'Nome': ['Ana', 'João', 'Maria'], 'Idade': [25, 30, 28]}
df = pd.DataFrame(data)

# Operações básicas
print(df)
print(df['Idade'].mean()) # 27.666...
```

3. Matplotlib

Explicação: Biblioteca para visualização de dados em gráficos.

```
import matplotlib.pyplot as plt

# Dados
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

# Criando um gráfico
plt.plot(x, y, marker='o')
plt.title('Gráfico Simples')
plt.xlabel('Eixo X')
plt.ylabel('Eixo Y')
plt.show()
```

4. Scikit-Learn

Explicação: Biblioteca para machine learning com algoritmos prontos para classificação, regressão, etc.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

# Carregando dados
iris = load_iris()
X, y = iris.data, iris.target

# Criando e treinando modelo
model = DecisionTreeClassifier()
model.fit(X, y)

# Fazendo previsão
print(model.predict([[5.1, 3.5, 1.4, 0.2]])) # [0]
```

5. TensorFlow

Explicação: Biblioteca para deep learning e computação numérica com redes neurais.

```
import tensorflow as tf

# Criando tensores
a = tf.constant([[1, 2], [3, 4]])
b = tf.constant([[5, 6], [7, 8]])

# Operação de matriz
result = tf.matmul(a, b)
print(result)
```

6. Keras

Explicação: API de alto nível para construção de redes neurais (geralmente usada com TensorFlow).

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Criando modelo sequencial
model = Sequential([
    Dense(10, activation='relu', input_shape=(4,)),
    Dense(3, activation='softmax')
])

# Compilando modelo
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy')
print(model.summary())
```

7. Flask

Explicação: Microframework para criar aplicações web simples.

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Olá, Flask!"

if __name__ == '__main__':
    app.run(debug=True)
```

8. Django

Explicação: Framework web completo para aplicações complexas.

```
# views.py em um projeto Django
from django.http import HttpResponse

def home(request):
    return HttpResponse("Olá, Django!")
```

9. Requests

Explicação: Biblioteca para fazer requisições HTTP.

```
import requests

response = requests.get('https://api.github.com') print(response.status_code) # 200
print(response.json()['current_user_url']) # 'https://api.github.com/user'
```

10. FastAPI

Explicação: Framework moderno para construir APIs rápidas com suporte a async.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Olá, FastAPI!"}

# Rodar com: uvicorn main:app --reload
```

Instruções:

Com base em cada item anterior faça uma pesquisa e crie um código simples, mas funcional para cada item.